

Ασκήσεις προετοιμασίας (2025-2026)

Κεφάλαιο 16 (Λογικός Προγραμματισμός - Prolog)

Ερώτημα 1

Έστω μια λογική πύλη AND με 2 εισόδους.

1. Γράψτε το κατηγορημα **and/3** με τα δύο πρώτα ορίσματα του (X και Y) να είναι οι εισόδοι (0 ή 1) και το τρίτο όρισμα (Z) να είναι η τιμή εξόδου (0 ή 1 αντίστοιχα).
2. Συνδυάζοντας 2 πύλες **and** κατασκευάστε ένα κύκλωμα που να δέχεται 3 εισόδους (X,Y,Z) και να επιστρέφει μια έξοδο (W) που θα είναι αληθής μόνο όταν και οι τρεις εισόδοι είναι αληθείς. Γράψτε το κατηγορημα **and3/4** χρησιμοποιώντας το κατηγορημα **and/3**.
3. Θέστε ερωτήματα που να επιστρέφουν για ποιες τιμές των εισόδων η έξοδος των πυλών είναι 0 τόσο για το κατηγορημα **and/3** όσο και για το κατηγορημα **and3/4**. Καταγράψτε και τα αποτελέσματα που επιστρέφουν τα ερωτήματα.

Απάντηση:

```
1.
and(0,0,0).
and(0,1,0).
and(1,0,0).
and(1,1,1).

2.
and3(X,Y,Z,W) :-
    and(X,Y,T),
    and(T,Z,W).

3.
?- and(X,Y,0).
X = 0, Y = 0 ;
X = 0, Y = 1 ;
X = 1, Y = 0.

?- and3(X,Y,Z,0).
X = 0, Y = 0, Z = 0 ;
X = 0, Y = 0, Z = 1 ;
X = 0, Y = 1, Z = 0 ;
X = 0, Y = 1, Z = 1 ;
X = 1, Y = 0, Z = 0 ;
X = 1, Y = 0, Z = 1 ;
X = 1, Y = 1, Z = 0.
```

Ερώτημα 2

Δίνεται η ακόλουθη βάση γνώσης:

```
parent(a,b).
parent(a,c).
parent(b,d).
parent(b,e).
parent(c,f).
descendent(X,Y) :- parent(Y,X).
descendent(X,Y) :- parent(Z,X), descendent(Z,Y).
```

Το κατηγορημα **parent(X,Y)** ερμηνεύεται ως ότι ο **X** είναι γονέας του **Y**, ενώ το κατηγορημα **descendent(X,Y)** ερμηνεύεται ως ότι ο **X** είναι απόγονος του **Y**.

1. Περιγράψτε λεκτικά το τι σημαίνουν οι δύο προτάσεις που συνιστούν το κατηγορήμα **descendent(X,Y)**.
2. Θέστε τα ακόλουθα ερωτήματα:
 - I. Είναι ο **f** απόγονος του **b**;
 - II. Ποιοι είναι οι απόγονοι του **a**;
 - III. Ποιοι είναι οι πρόγονοι του **f**;

Απάντηση:

1.
Η πρώτη πρόταση σημαίνει ότι αν ο Y είναι γονέας του X τότε συνεπάγεται ότι ο X είναι απόγονος του Y.
Η δεύτερη πρόταση σημαίνει ότι αν ο Z είναι γονέας του X και ο Z είναι απόγονος του Y τότε συνεπάγεται ότι ο X είναι απόγονος του Y.
- 2.I
?- descendent(f, b).
false.
- 2.II
?- descendent(X, a).
- 2.III
?- descendent(f, X).

Ερώτημα 3

Ποια θα είναι τα αποτελέσματα των ακόλουθων ερωτημάτων;

1. ?- [H|T] = [1,2,3].
2. ?- [_,X,_|T] = [1,2,3,4,5].
3. ?- X = 1, Y is X + 1.
4. ?- member(X,[1,2,3]).
5. ?- member(X,[a,b]), member(Y,[1,2]).
6. ?- X = 1, Y = X + 1.
7. ?- X = 1 + 2, 1 + 2 = Y.
8. ?- X = 1, X = 2.
9. ?- X is Y + 2, Y = 2.
10. ?- append(L1,L2,[1,2,3]).

Απάντηση:

1.
H = 1,
T = [2, 3].
2.
X = 2,
T = [4, 5].
3.
X = 1,
Y = 2
4.
member(X,[1,2,3]).
X = 1 ;
X = 2 ;

X = 3.

5.

X = a, Y = 1 ;

X = a, Y = 2 ;

X = b, Y = 1 ;

X = b, Y = 2.

6.

X = 1,

Y = 1+1.

7.

X = Y, Y = 1+2.

8.

false.

9.

ERROR: Arguments are not sufficiently instantiated

10.

L1 = [],

L2 = [1, 2, 3] ;

L1 = [1],

L2 = [2, 3] ;

L1 = [1, 2],

L2 = [3] ;

L1 = [1, 2, 3],

L2 = [] ;

false.

Ερώτημα 4

Δίνεται ο ακόλουθος ορισμός για το κατηγορήμα append/3:

```
append([], List, List).  
append([Head | List_1], List_2, [Head | List_3]) :-  
    append(List_1, List_2, List_3).
```

1. Περιγράψτε τη λειτουργία του κατηγορήματος append/3.
2. Δώστε 3 παραδείγματα όπου η χρήση του κατηγορήματος append/3 γίνεται με διαφορετικό τρόπο.
3. Εξηγήστε την υλοποίησή του append/3, αναλύοντας τις 2 προτάσεις από τις οποίες αποτελείται.

Απάντηση:

1.
Το κατηγορήμα append/3 χρησιμοποιείται για τη συνένωση 2 λιστών σε 1 τρίτη λίστα. Επίσης μπορεί να χρησιμοποιηθεί για τη διάσπαση μιας λίστας σε 2 μέρη ή για να ελεγχθεί αν μια λίστα είναι το τέλος μιας άλλης λίστας.
2.
Μερικά παραδείγματα χρήσης του append/3 είναι:
α) Συνένωση 2 λιστών σε 1 άλλη λίστα
?- append([1,2], [3,4], L).
L = [1,2,3,4].

β) Όλοι οι πιθανοί τρόποι διαχωρισμού μιας λίστας σε 2 επιμέρους λίστες

?- append(L1,L2, [a,b]).

L1 = [],

L2 = [a, b] ;

L1 = [a],

L2 = [b] ;

L1 = [a, b],

L2 = [] ;

false.

γ) Έλεγχος αν μια λίστα εφόσον προσαρτηθεί σε μια άλλη θα δώσει μια τρίτη λίστα.

?- L = [4,5], append([1,2,3],L,[1,2,3,4,5]).

L = [4, 5].

3.

Η πρώτη πρόταση (βασική περίπτωση της αναδρομής) σημαίνει ότι αν η πρώτη παράμετρος του κατηγορήματος append/3 είναι η κενή λίστα τότε η τρίτη παράμετρος θα είναι ίση με τη δεύτερη παράμετρο.

Η δεύτερη πρόταση (αναδρομική περίπτωση) σημαίνει ότι αν η πρώτη παράμετρος είναι μια λίστα με πρώτο στοιχείο το Head, τότε η τρίτη παράμετρος θα είναι μια λίστα με πρώτο στοιχείο το Head που θα ακολουθείται από μια λίστα (την List_3) που θα προκύπτει ως προσάρτηση της List_2 στη List_1.

Ερώτημα 5

Γράψτε το κατηγορήμα product_list/2 που θα επιστρέφει το γινόμενο των στοιχείων μιας λίστας. Για παράδειγμα:

?- product_list([1,2,3], X).

X = 6.

Απάντηση:

% Βασική περίπτωση: Το γινόμενο της άδειας λίστα είναι 1.

product_list([], 1).

% Αναδρομική περίπτωση: Το γινόμενο της λίστας είναι το γινόμενο του

% πρώτου στοιχείου με το γινόμενο της υπόλοιπης λίστας.

product_list([Head | Tail], Product) :-

product_list(Tail, TailProduct),

Product is Head * TailProduct.

Ερώτημα 6

Δίνεται η ακόλουθη βάση γνώσης σε Prolog:

woman(helen).

woman(maria).

woman(sofia).

loves(nikos, helen). % ο Νίκος αγαπά την Ελένη

loves(panos, helen).

loves(petros, viki).

loves(viki, petros).

jealous(X,Y):-

loves(X,Z),

loves(Y,Z).

1. Πόσες προτάσεις, πόσα γεγονότα και πόσους κανόνες περιέχει η βάση γνώσης;

2. Ποια κατηγορήματα, ποιους ατομικούς όρους και ποιες μεταβλητές περιέχει η βάση γνώσης;

3. Ερμηνεύστε λεκτικά την τελευταία πρόταση της βάσης γνώσης.

4. Αν η βάση γνώσης βρίσκεται σε ένα αρχείο με όνομα prolog_erotima6.pl, πως μπορεί να φορτωθεί στη γραμμή εντολών, από το swipl.

5. Ποια θα είναι τα αποτελέσματα εκτέλεσης των ακόλουθων ερωτημάτων;
- I. ?- woman(X).
 - II. ?- loves(panos,X), woman(X).
 - III. ?- loves(petros,X), woman(X).
 - IV. ?- jealous(panos,X).

Απάντηση:

1.
Η βάση γνώσης περιέχει 8 προτάσεις από τις οποίες οι 7 είναι γεγονότα και 1 είναι κανόνας.

2.
Τα κατηγορήματα είναι τα: woman/1, loves/2 και jealous/2
Οι ατομικοί όροι είναι οι: helen, maria, sofia, viki, petros
Οι μεταβλητές που αναφέρονται στη βάση γνώσης είναι οι: X, Y, Z

3.
Η τελευταία πρόταση της βάσης γνώσης είναι κανόνας και σημαίνει ότι αν ο X αγαπά τον Z και ο Y αγαπά τον Z τότε ο X ζηλεύει τον Y.

4.
Ένας τρόπος φόρτωσης της βάσης γνώσης, δεδομένου ότι βρισκόμαστε στον ίδιο φάκελο με το αρχείο prolog_erotima6.pl, είναι γράφοντας:

```
$ swipl -l prolog_erotima6.pl
```

Εναλλακτικά η φόρτωση της βάσης γνώσης μπορεί να γίνει ως εξής:

```
$ swipl
?- consult('prolog_erotima6.pl').
```

5.I
?- woman(X).
X = helen ;
X = maria ;
X = sofia.

5.II
?- loves(panos,X), woman(X).
X = helen.

5.III
?- loves(petros,X), woman(X).
false.

5.IV
?- jealous(panos,X).
X = nikos ;
X = panos.

Ερώτημα 7

Για τον ακόλουθο κώδικα σε Prolog, δώστε ερμηνεία σε φυσική γλώσσα των ακόλουθων προτάσεων.

Τι είδους πρόταση είναι η κάθε μια;

- 1. likes(nikos, maria).
- 2. friends(X, Y) :- likes(X, Y), likes(Y, X).

3. ?- friends(nikos, X).

Απάντηση:

1. Στον Νίκο αρέσει η Μαρία. (γεγονός=fact)
2. Αν στον X αρέσει ο Y και στον Y αρέσει ο X τότε ο X είναι φίλος του Y. (κανόνας=rule)
3. Ποιοι είναι οι φίλοι του Νίκου; (ερώτημα=query)

Ερώτημα 8

A) Δίνεται το ακόλουθο κατηγορημα `product_from_to_by_step/4` που υπολογίζει το γινόμενο των ακέραιων τιμών από μια αρχική τιμή (Start) μέχρι μια τελική τιμή (End) με ένα θετικό βήμα (Step). Εξηγήστε την λειτουργία του.

```
product_from_to_by_step(Start, End, _, 1) :-  
    Start > End.  
  
product_from_to_by_step(Start, End, Step, Product) :-  
    Start <= End,  
    Next is Start + Step,  
    product_from_to_by_step(Next, End, Step, SubProduct),  
    Product is Start * SubProduct.
```

B) Γράψτε το ερώτημα που επιστρέφει το γινόμενο $2*4*6*8*10$.

Απάντηση:

A) Το κατηγορημα `product_from_to_by_step(Start, End, Step, Product)` υπολογίζει το γινόμενο των τιμών από το Start μέχρι το End με βήμα Step. Το κατηγορημα αποτελείται από δύο κανόνες.

Ο πρώτος κανόνας είναι η βασική περίπτωση. Αν η αρχική τιμή Start έχει ξεπεράσει την τελική τιμή End, τότε δεν υπάρχει άλλος αριθμός για πολλαπλασιασμό. Σε αυτή την περίπτωση το αποτέλεσμα είναι 1, δηλαδή το ουδέτερο στοιχείο του πολλαπλασιασμού.

Ο δεύτερος κανόνας είναι η αναδρομική περίπτωση. Αν $Start \leq End$, τότε η τρέχουσα τιμή Start συμμετέχει στο γινόμενο. Υπολογίζεται η επόμενη τιμή Next που χρησιμοποιείται στην αναδρομική κλήση. Στη συνέχεια, το κατηγορημα καλείται αναδρομικά για τον υπολογισμό του γινομένου των επόμενων τιμών και τελικά το αποτέλεσμα προκύπτει πολλαπλασιάζοντας το Start με το SubProduct.

B)

?- `product_from_to_by_step(2,10,2, X)`.

Ερώτημα 9

Δίνεται η ακόλουθη βάση γνώσης:

```
% student(StudentName).
```

```
student(alice).
```

```
student(bob).
```

```
student(charlie).
```

```
% course(CourseCode, CourseName).
```

```
course(cs101, 'Intro to CS').
```

```
course(cs102, 'Data Structures').
```

```
course(cs103, 'Logic Programming').
```

```
% grade(Student, Course, Grade).
```

```
grade(alice, cs101, 85).
```

```
grade(alice, cs102, 90).
```

```
grade(bob, cs101, 70).
grade(bob, cs103, 78).
grade(charlie, cs101, 95).
grade(charlie, cs102, 88).
```

A. Γράψτε τα κατηγορήματα:

- `passed(Student, Course, PassGrade)` που να επιστρέφει τα ονόματα των φοιτητών και τους κωδικούς μαθημάτων στα οποία έχουν βαθμό πάνω από `PassGrade`.
- `enrolled(Student, Course)` που να επιστρέφει ονόματα φοιτητών και μαθημάτων στα οποία υπάρχει βαθμός

B. Γράψτε τα ερωτήματα:

- Τι βαθμό πήρε η `alice` στο μάθημα 'Intro to CS';
- Ποια είναι τα ονόματα φοιτητών και μαθημάτων με βαθμό πάνω από 75;
- Ποιοι είναι οι εγγεγραμμένοι φοιτητές στο μάθημα 'Logic Programming';

Απάντηση:

A)

```
passed(Student, Course, PassGrade):-
    student(Student),
    course(Course, _),
    grade(Student, Course, Grade),
    Grade >= PassGrade.

enrolled(Student, Course) :-
    student(Student),
    grade(Student, Course, _).
```

B)

```
?- grade(alice, cs101, G).
G = 85.

?- passed(S,C,75).
S = alice,
C = cs101 ;
S = alice,
C = cs102 ;
S = bob,
C = cs103 ;
S = charlie,
C = cs101 ;
S = charlie,
C = cs102 ;
false.

?- enrolled(S, cs103).
S = bob ;
false.
```

Ερώτημα 10

A) Γράψτε το κατηγορήμα `product\4` που να πολλαπλασιάζει 3 αριθμούς και να επιστρέφει το αποτέλεσμα.

B) Γράψτε 2 διαφορετικούς τρόπους με τους οποίους μπορεί να χρησιμοποιηθεί σε ερωτήματα.

Απάντηση:

A)

```
product(A, B, C, Result) :-  
    Result is A * B * C.
```

B)

Υπολογισμός του γινομένου $2 * 3 * 4$

```
?- product(2, 3, 4, X).  
X = 24.
```

Έλεγχος αν το γινόμενο $2 * 3 * 4$ είναι ίσο με 30

```
?- product(2, 3, 4, X).  
X = 24.  
  
?- product(2, 3, 4, 30).  
false.
```

Ερώτημα 11

Τι πρόκειται να εμφανίσει η εκτέλεση των ακόλουθων ερωτημάτων;

1. `?- member(X, [1,2,3]), member(Y, [1,2,3]), X \= Y.`
2. `?- A = 1, B = 2, C = A + B.`
3. `?- A = 1, B = 2, C is A + B.`
4. `?- X = 3, Y = X, Y = 4.`
5. `?- member(X, [a,b,c]), X \= b.`
6. `?- append([1,2], [3,4], L).`
7. `?- append(A, B, [1,2,3]).`
8. `?- X is 2 + 3 * 4, X > 10.`

Απάντηση:

1.
`X = 1, Y = 2 ;`
`X = 1, Y = 3 ;`
`X = 2, Y = 1 ;`
`X = 2, Y = 3 ;`
`X = 3, Y = 1 ;`
`X = 3, Y = 2.`
2.
`A = 1, B = 2, C = 1 + 2.`
3.
`A = 1, B = 2, C = 3.`
4.
`False.`
5.
`X = a ;`
`X = c.`
6.
`L = [1, 2, 3, 4].`
7.
`A = [], B = [1, 2, 3] ;`
`A = [1], B = [2, 3] ;`
`A = [1, 2], B = [3] ;`


```
A = [1, 2, 3], B = [].
```

```
8.
```

```
X = 14.
```

Ερώτημα 12

A) Γράψτε το κατηγορήμα `find_pairs/2` που να εντοπίζει σε μια λίστα με ζεύγη, το ζεύγος ή τα ζεύγη που έχουν την τιμή που θα δίνεται ως πρώτο όρισμα. Για παράδειγμα για τα ερωτήματα:

```
?- find_pairs((2,3), [(1,2),(2,3),(55,21)]).
```

```
?- find_pairs((55, X), [(1,2),(2,3),(55,21)]).
```

θα πρέπει να επιστρέφει αντίστοιχα:

```
true.
```

```
X=21.
```

B) Τι θα επιστρέψει το ακόλουθο ερώτημα:

```
?- find_pairs(X, [(1,2), (3,4), (5,6)]).
```

Απάντηση:

A)

```
find_pairs((X,Y), [(X,Y)| _]).  
find_pairs((X,Y), [_ | T]) :-  
    find_pairs((X,Y), T).
```

B)

```
?- find_pairs(X, [(1,2), (3,4), (5,6)]).  
X = (1,2);  
X = (3,4);  
X = (5,6);  
False.
```

Ερώτημα 13

Δίνεται ο ακόλουθος ορισμός για το κατηγορήμα `foo/2` στη Prolog:

```
foo([], 0).  
foo([_ | T], L) :-  
    foo(T, TL),  
    L is TL + 1.
```

A) Περιγράψτε τη λειτουργία του κατηγορήματος `foo/2`.

B) Δώστε 2 παραδείγματα όπου η χρήση του `foo/2` μπορεί να γίνει με διαφορετικό τρόπο.

Γ) Εξηγήστε την υλοποίηση του `foo/2`, αναλύοντας τις 2 προτάσεις από τις οποίες αποτελείται.

Απάντηση:

A)

Υπολογίζει το μήκος μιας λίστας.

B)

Υπολογισμός του μήκους μιας λίστας.

```
?- foo([1,2,3],X).
```

```
X= 3
```

Έλεγχος ότι η λίστα `[1,2,3]` έχει μήκος 3.

```
?- foo([1,2,3],3).
```

true.

Γ)

Αν η λίστα είναι κενή τότε το μήκος της είναι 0.

Αν η λίστα δεν είναι κενή τότε το μήκος της είναι όσο το μήκος της ουράς της λίστας συν 1.

Ερώτημα 14

A) Να υλοποιήσετε το κατηγορημα dot_product/3, το οποίο δέχεται δύο λίστες αριθμών ίδιου μήκους και υπολογίζει το εσωτερικό τους γινόμενο. Το αποτέλεσμα να επιστρέφεται ως τρίτο όρισμα.

Παρατήρηση: Το εσωτερικό γινόμενο δύο λιστών $[A_1, A_2, \dots, A_n]$ και $[B_1, B_2, \dots, B_n]$ ορίζεται ως:

$$A_1 \times B_1 + A_2 \times B_2 + \dots + A_n \times B_n$$

B) Έστω ότι ο κώδικας του προηγούμενου ερωτήματος έχει αποθηκευτεί σε ένα αρχείο με όνομα exams.pl. Περιγράψτε τη διαδικασία που πρέπει να ακολουθηθεί για τη φόρτωση του κώδικα στο περιβάλλον της SWI-Prolog από τη γραμμή εντολών και την εντολή με την οποία θα κληθεί το κατηγορημα dot_product/3 με τις λίστες [1,2,3] και [4,5,6] και ποιο θα είναι το αποτέλεσμα που θα επιστραφεί από την Prolog.

Απάντηση:

A)

```
dot_product([], [], 0).  
dot_product([X|Xs], [Y|Ys], Result) :-  
    dot_product(Xs, Ys, R1),  
    Result is R1 + X * Y.
```

B)

```
$ swipl  
?- [exams].  
  
?- dot_product([1,2,3],[4,5,6],X)  
X=32.
```

Ερώτημα 15

Ορίστε το κατηγορημα min/3 που επιστρέφει τον ελάχιστο από δύο αριθμούς.

Απάντηση:

```
min(X, Y, X) :- X =< Y.  
min(X, Y, Y) :- X > Y.
```

Ερώτημα 16

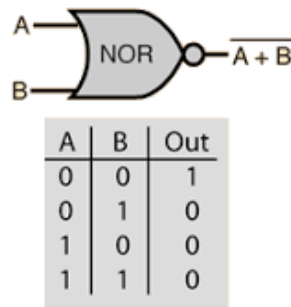
Ορίστε το κατηγορημα same_start_end/1 που να ελέγχει αν μια λίστα έχει ίδιο πρώτο και τελευταίο στοιχείο.

Απάντηση:

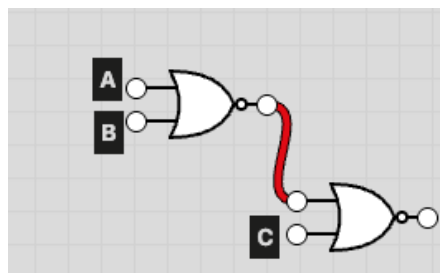
```
same_start_end([X|Xs]) :-  
    last_element(Xs, X).  
  
last_element([Y], Y).  
last_element(_|T, Y) :-  
    last_element(T, Y).
```

Ερώτημα 17

A) Στην Prolog γράψτε το κατηγορημα `nor/3` με τα 2 πρώτα ορίσματα να είναι οι είσοδοι μιας πύλης NOR και το τρίτο να είναι η έξοδος της πύλης. Δίνεται ο ακόλουθος πίνακας αληθείας για την πύλη NOR:



B) Χρησιμοποιώντας το κατηγορημα `nor/3` γράψτε το νέο κατηγορημα `nor/4` σύμφωνα με το σχήμα που ακολουθεί:



Γ) Αν ο κώδικας για τα κατηγορήματα `nor/3` και `nor/4` έχει αποθηκευτεί σε ένα αρχείο με όνομα `exams.pl` με ποιο τρόπο θα φορτωθεί στην SWI-Prolog; Παραθέστε τις εντολές που χρειάζονται.

Δ) Γράψτε τα ακόλουθα ερωτήματα στην Prolog:

1. Ποια είναι η τιμή της εξόδου στην πύλη NOR με 2 εισόδους, όταν οι είσοδοι είναι 1 και 1;
2. Για ποιες τιμές της εισόδου, η έξοδος είναι 0 στην πύλη NOR με 3 εισόδους;

Απάντηση:

A)

```
% nor(A, B, Output) υλοποιεί την λογική πύλη NOR
nor(0, 0, 1).
nor(0, 1, 0).
nor(1, 0, 0).
nor(1, 1, 0).
```

B)

```
nor(A, B, C, Output) :-
    nor(A, B, Temp),      % πρώτα κάνουμε NOR στα A και B
    nor(Temp, C, Output). % έπειτα NOR του αποτελέσματος με C
```

Γ)

```
$ swipl
```

```
$ [exams].
```

```
ή
```

```
$ consult(exams.pl).
```

Δ)

```
?- nor(1, 1, Output).
?- nor(A, B, C, 0).
```

Ερώτημα 18

A) Δίνεται η ακόλουθη βάση γνώσης στην Prolog:

```
movie('Inception', 'Nolan', 2010).
movie('Interstellar', 'Nolan', 2014).
movie('Pulp Fiction', 'Tarantino', 1994).
movie('Django Unchained', 'Tarantino', 2012).
movie('The Matrix', 'Wachowski', 1999).
```

Γράψτε τα Prolog κατηγορήματα:

1. **directed_by(Director, Title)**, που να επιστρέφει τους τίτλους ταινιών ενός σκηνοθέτη.
2. **before_year(Year, Title)**, που να επιστρέφει τους τίτλους ταινιών που έχουν κυκλοφορήσει πριν από μια ημερομηνία.
3. **directed_by_before_year(Director, Year, Title)**, που να επιστρέφει τους τίτλους ταινιών ενός σκηνοθέτη που έχουν κυκλοφορήσει πριν από μια ημερομηνία.

B) Θέστε το ερώτημα: Ποιες είναι οι ταινίες που έχουν σκηνοθετηθεί από τον σκηνοθέτη Tarantino πριν το 2000;

Απάντηση:

A)

```
1.
directed_by(Director, Title) :-
    movie(Title, Director, _).

2.
before_year(Year, Title) :-
    movie(Title, _, MovieYear),
    MovieYear < Year.

3.
directed_by_before_year(Director, Year, Title)
directed_by_before_year(Director, Year, Title) :-
    movie(Title, Director, MovieYear),
    MovieYear < Year.
```

B)

```
?- directed_by_before_year('Tarantino', 2000, Title).
```

Ερώτημα 19

Δίνεται ο ακόλουθος ορισμός για το κατηγορήμα nth0/3 της Prolog:

```
nth0(0, [Element|_], Element).
nth0(Index, [_|Tail], Element) :-
    Index > 0,
    NewIndex is Index - 1,
    nth0(NewIndex, Tail, Element).
```

1. Περιγράψτε τη λειτουργία του κατηγορήματος nth0/3.
2. Δώστε 2 παραδείγματα όπου η χρήση του nth0/3 μπορεί να γίνει με διαφορετικό τρόπο.
3. Εξηγήστε την υλοποίηση του nth0/3.

Απάντηση:

1. Το κατηγόρημα `nth0/3` βρίσκει το στοιχείο μιας λίστας που βρίσκεται σε μια συγκεκριμένη θέση (η οποία ξεκινά από το 0). Το πρώτο όρισμα είναι η θέση (`index`), το δεύτερο είναι η λίστα, και το τρίτο είναι το στοιχείο της λίστας που βρίσκεται στη συγκεκριμένη θέση.

2.

```
?- nth0(2, [a, b, c, d], Element).  
Element = c.
```

```
?- nth0(Index, [a, b, c], c).  
Index = 2.
```

3.

Το κατηγόρημα λειτουργεί αναδρομικά, μειώνοντας τον δείκτη κάθε φορά και εξετάζοντας την ουρά της λίστας, μέχρι να βρεθεί το στοιχείο που αντιστοιχεί στον αρχικό δείκτη.

Ερώτημα 20

Δίνεται ο ακόλουθος ορισμός για το κατηγόρημα `member/2` στην Prolog:

```
member(X, [X|_]).  
member(X, [_|_]) :- member(X, _).
```

1. Περιγράψτε τη λειτουργία του κατηγορήματος `member/2`.
2. Δώστε 2 παραδείγματα όπου η χρήση του κατηγορήματος `member/2` γίνεται με διαφορετικό τρόπο.
3. Εξηγήστε την υλοποίηση του `member/2`, αναλύοντας τις 2 προτάσεις από τις οποίες αποτελείται.

Απάντηση:

1. Το κατηγόρημα `member/2` εξετάζει αν ένα στοιχείο εντοπίζεται μέσα σε μια λίστα.

2.

```
?- member(1, [1,2,3]).  
true.
```

```
?- member(X, [1,2,3]).  
X = 1;  
X = 2;  
X = 3.
```

3.

Η πρώτη πρόταση σημαίνει ότι το `X` είναι μέλος της λίστας αν είναι η κεφαλή της λίστας.

Η δεύτερη πρόταση είναι αναδρομική και αναφέρει ότι το `X` είναι μέλος της λίστας, αν είναι μέλος της ουράς της λίστας.